

Python Programming

Table of Contents

Topic 1: Overview of Python Programming.....	2
Topic 2: Integrated Development Environments for Python.....	3
Topic 3: Fundamentals of Variables and Data Types.....	5
Topic 4: Numbers, Strings, List, Tuple, Dictionary.....	6
Topic 5: Operators - Arithmetic, Comparison, Assignment, Logical, Bitwise Operators.....	10
Topic 6: Decision Making.....	13
Topic 7: Looping in Python.....	14
Topic 8: Conversion Functions (int(), long(), float()).....	15
Topic 9: Mathematical Operations and Functions.....	17
Topic 10: Strings - Escape Characters, String Special Operators, String Formatting Operators.....	19
Topic 11: Built-in String Methods.....	20
Topic 12: Python Lists - Working with Lists.....	22
Topic 13: Built-in Functions - cmp(), len(), min(), max(), list comprehension.....	23
Topic 14: Tuples - Accessing Values in Tuples, Deleting Tuple Elements, Indexing, Slicing, and Matrices.....	24
Topic 15: Built-in Tuple Functions - cmp(), len().....	25
Topic 16: Dictionaries - Accessing Values from Dictionaries, Deleting and Updating Elements in Dictionaries.....	26
Topic 17: Properties of Dictionaries, Built-in Dictionary Functions & Methods, Dictionary Comprehension.....	27
Topic 18: Date & Time - Time Tuple, Calendar Module, and Time Module.....	29
Topic 19: Functions - Define Function, Calling Function, Pass by Reference as Value, Function Arguments, Anonymous Functions, Return Statements.....	31
Topic 20: Scope of Variables - Local & Global, Decorators, and Recursion.....	34
Topic 21: Map, Reduce, and Filter.....	35
Topic 22: Import Statements, Locating Modules - Current Directory, Pythonpath.....	36
Topic 23: Dir() Function, Global and Location Functions, Reload() Function, Sys Module, and Subprocess Module.....	38
Topic 24: Packages in Python.....	40
Topic 25: Files in Python - Reading Keyboard Input, Input Function, Opening and Closing Files, Syntax, and List of Modes.....	41
Topic 26: Files Object Attributes - open, close, Reading and Writing Files, File Position.....	43
Topic 27: Renaming and Deleting Files.....	44
Topic 28: Pickle and JSON.....	45
Topic 29: mkdir Method, chdir() Method, getcwd Method, rmdir.....	47
Topic 30: Exception Handling - List of Exceptions, Try and Except.....	49

Exception Handling.....	49
Topic 31: Try-Finally Clause and User-Defined Exceptions.....	50
Topic 32: Object-Oriented Programming (OOP) Concepts, Class, Objects, Inheritance, Overriding Methods, Overloading Operators, Data Hiding.....	52
Topic 33: match Function, search Function, Matching vs. Searching.....	55
Topic 35: Database Connectivity - Create, Insert, Update, and Delete Operations, Handling Errors.....	56
Topic 34: Regular Expression Modifiers and Patterns.....	59

Topic 1: Overview of Python Programming

Python Introduction

- Python is a high-level, interpreted programming language known for its simplicity and readability.
- It was created by Guido van Rossum and first released in 1991.

Key Features

Simplicity and Readability:

- Python's syntax is clear and concise, promoting readability.
- Uses indentation for block structure, enhancing code aesthetics.

Dynamic Typing:

- Dynamically typed language; variable types are determined at runtime.
- Offers flexibility but maintains strict type checking.

Extensive Standard Library:

- Python comes with a comprehensive standard library.
- Includes modules for various tasks, reducing the need for external libraries.

Interpreted Language:

- Python code is executed line by line, without a separate compilation step.
- Facilitates rapid development and testing.

Cross-Platform Compatibility:

- Code written in Python is platform-independent.
- Runs on Windows, macOS, and Linux without modification.

Automatic Memory Management:

- Features a built-in garbage collector for automatic memory management.
- Eliminates the need for manual memory allocation and deallocation.

Versatility:

- Widely used across industries and domains.
- Applications include web development, data science, machine learning, automation, and more.

Programming Cycle of Python

1. Writing Code:

- You create Python programs by writing code using a text editor or integrated development environment (IDE).

2. Compilation:

- Python is an interpreted language, so it doesn't require a separate compilation step.
- The code is executed line by line.

3. Execution:

- Python code is executed by the Python interpreter.
- The interpreter reads your code, processes it, and produces output.

4. Debugging:

- Debugging is the process of identifying and fixing errors (bugs) in your code.
- Python provides tools like `print()` statements and debugging libraries for this purpose.

5. Testing:

- You should test your Python code thoroughly to ensure it works as expected.
- Unit tests, integration tests, and functional tests are common testing approaches.

6. Documentation:

- Writing clear and concise documentation is essential to help others understand your code.
- Tools like docstrings and comments are used for documentation.

7. Maintenance:

- Software maintenance involves updates, bug fixes, and improvements to the code.
- Python's simplicity can make maintenance easier.

8. Deployment:

- Deploying Python applications can vary, but it often involves packaging your code and its dependencies for distribution.

Example:

```
# Sample Python code
def greet(name):
    """This function greets the person passed in as a parameter."""
    print(f"Hello, {name}!")

# Calling the function
greet("Alice")
```

In this code, we define a function `greet()` and then call it to print a greeting to a person's name.

Topic 2: Integrated Development Environments for Python

Python IDE (Integrated Development Environment)

- An Integrated Development Environment (IDE) is a software application that provides comprehensive facilities for Python programming.

Popular Python IDEs:

1. **PyCharm:** Developed by JetBrains, PyCharm is a popular and feature-rich IDE for Python with excellent debugging and code analysis capabilities.
2. **Visual Studio Code (VS Code):** A free, open-source code editor with a wide range of extensions and excellent Python support. It's highly customizable.
3. **Jupyter Notebook:** Great for data science and machine learning, Jupyter Notebook is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations, and narrative text.
4. **IDLE (Integrated Development and Learning Environment):** IDLE comes bundled with Python and is a simple IDE that's suitable for beginners.
5. **Spyder:** An open-source IDE specifically designed for scientific computing and data analysis with Python.

Features of Python IDEs:

- **Code Editor:** Most IDEs offer a code editor with features like syntax highlighting, code completion, and error checking.

- **Debugging Tools:** Debugging is made easier with built-in debuggers, breakpoints, and variable inspection.
- **Project Management:** IDEs allow you to organize your code into projects and provide tools for managing project files.
- **Version Control Integration:** Many IDEs support version control systems like Git for collaborative development.
- **Extensions:** IDEs often support extensions and plugins to enhance functionality based on your needs.

Example (Using Visual Studio Code):

1. Download and install Visual Studio Code.
2. Install the "Python" extension from the Visual Studio Code marketplace.
3. Create a Python file (e.g., `hello.py`) and start writing Python code.
4. Use features like auto-completion, linting, and debugging to enhance your coding experience.

Remember that the choice of IDE depends on your specific needs and preferences. Each IDE has its own strengths and weaknesses.

Topic 3: Fundamentals of Variables and Data Types

Variables

- In Python, variables are used to store data values.
- Variables are created when you assign a value to them.
- Variable names are case-sensitive and can consist of letters, numbers, and underscores but must start with a letter or underscore.
- Examples:

```
x = 5 # x is a variable storing the integer 5
name = "John" # name is a variable storing a string
is_valid = True # is_valid is a variable storing a boolean
```

Data Types

Python has several built-in data types, including:

1. **int:** Integer data type for whole numbers.

```
age = 25
```

2. **float**: Floating-point data type for numbers with decimal points.

```
price = 12.99
```

3. **str**: String data type for text.

```
name = "Alice"
```

4. **bool**: Boolean data type for True or False values.

```
is_valid = True
```

5. **list**: A collection of values, ordered and changeable.

```
fruits = ["apple", "banana", "cherry"]
```

6. **tuple**: A collection of values, ordered and immutable.

```
dimensions = (10, 20, 15)
```

7. **dict**: A collection of key-value pairs.

```
person = {"name": "John", "age": 30}
```

8. **set**: A collection of unique values, unordered.

```
unique_numbers = {1, 2, 3, 4}
```

Example (Variables and Data Types):

```
# Variable assignments
age = 25
name = "Alice"
is_valid = True

# Data types
print(type(age)) # <class 'int'>
print(type(name)) # <class 'str'>
print(type(is_valid)) # <class 'bool'>
```

In this example, we define variables with different data types and check their types using the `type()` function.

Topic 4: Numbers, Strings, List, Tuple, Dictionary

Numbers

Python supports various types of numbers, including integers (`int`) and floating-point numbers (`float`).

```
# Integer
x = 5

# Floating-point
y = 3.14
```

Strings:

Strings are sequences of characters enclosed in single or double quotes.

String Types:

```
name = "Alice"
```

In Python, there are several types of strings, including:

1. **Regular Strings:** These are the most common type of strings and are defined using single quotes (`'`) or double quotes (`"`). For example:

```
string1 = 'This is a regular string.'

string2 = "This is also a regular string."
```

2. **Multiline Strings:** Multiline strings are used when you want to define strings that span multiple lines. They are typically enclosed in triple single quotes (`'''`) or triple double quotes (`"""`). For example:

```
multiline_string = '''This is a
multiline string
```

```
spanning multiple lines. '''
```

3. **Raw Strings:** Raw strings are used when you want to treat backslashes (`\`) as literal characters rather than escape characters. They are defined by prefixing the string with an `r`. For example:

```
raw_string = r'This is a raw string with \n newline.'
```

4. **Unicode Strings:** Unicode strings are used to work with Unicode characters and are defined by adding a `u` or `U` prefix before the string. For example:

```
unicode_string = u'This is a Unicode string: \u0394'
```

5. **Byte Strings:** Byte strings are used to store binary data and are defined by adding a `b` or `B` prefix before the string. For example:

```
byte_string = b'This is a byte string'
```

6. **Formatted Strings (f-strings):** As explained earlier, f-strings are used to embed expressions and variables within strings by prefixing the string with an `f`. For example:

```
name = 'Alice'

formatted_string = f'Hello, {name}!'
```

7. **Bytes Objects:** Bytes objects are similar to byte strings but are created as a sequence of bytes, typically represented as integers in the range 0-255. They are created using the `bytes()` constructor. For example:

```
byte_data = bytes([65, 66, 67, 68]) # Creates a bytes object with ASCII values
of 'A', 'B', 'C', 'D'
```

These are the most common types of strings in Python, each with its own specific use cases and characteristics. You can choose the type of string that best suits your needs depending on the data you are working with and the requirements of your code.

List

A list is an ordered collection of items. Lists are mutable (can be modified).

```
fruits = ["apple", "banana", "cherry"]
```

Tuple

A tuple is an ordered collection of items, but unlike lists, they are immutable (cannot be modified).

```
dimensions = (10, 20, 15)
```

Dictionary

A dictionary is an unordered collection of key-value pairs.

```
person = {"name": "John", "age": 30}
```

Accessing Elements

You can access elements of lists, tuples, and dictionaries using indexing.

```
# Access list element
first_fruit = fruits[0]

# Access tuple element
width = dimensions[0]

# Access dictionary value by key
person_name = person["name"]
```

Modifying Lists

You can modify lists by adding, removing, or changing elements.

```
# Add an item to a list
fruits.append("orange")

# Remove an item from a list
fruits.remove("banana")

# Change a list item
fruits[0] = "kiwi"
```

Immutable Tuples

Tuples are immutable, so you cannot change their elements once defined.

```
# This will raise an error
dimensions[0] = 12
```

Dictionary Operations

Dictionaries allow you to update, add, or delete key-value pairs.

```
# Update a value
person["age"] = 31

# Add a new key-value pair
person["city"] = "New York"

# Delete a key-value pair
del person["age"]
```

Topic 5: Operators - Arithmetic, Comparison, Assignment, Logical, Bitwise Operators

Arithmetic Operators

Arithmetic operators perform mathematical operations on numbers.

- Addition **+**: Adds two numbers.
- Subtraction **-**: Subtracts the right operand from the left operand.
- Multiplication *****: Multiplies two numbers.

- Division `/`: Divides the left operand by the right operand.
- Modulus `%`: Returns the remainder of division.
- Exponentiation `**`: Raises the left operand to the power of the right operand.
- Floor Division `//`: Returns the quotient of division, discarding the remainder.

```
x = 10
y = 3

addition = x + y
subtraction = x - y
multiplication = x * y
division = x / y
remainder = x % y
exponentiation = x ** y
floor_division = x // y
```

Comparison Operators

Comparison operators are used to compare two values.

- Equal to `==`: Returns `True` if both operands are equal.
- Not equal to `!=`: Returns `True` if operands are not equal.
- Greater than `>`: Returns `True` if the left operand is greater than the right operand.
- Less than `<`: Returns `True` if the left operand is less than the right operand.
- Greater than or equal to `>=`: Returns `True` if the left operand is greater than or equal to the right operand.
- Less than or equal to `<=`: Returns `True` if the left operand is less than or equal to the right operand.

```
a = 5
b = 7

equal = a == b
not_equal = a != b
greater = a > b
less = a < b
greater_equal = a >= b
less_equal = a <= b
```

Assignment Operators

Assignment operators are used to assign values to variables.

- Assignment `=`: Assigns the value on the right to the variable on the left.
- Addition assignment `+=`: Adds the right operand to the left operand and assigns the result to the left operand.
- Subtraction assignment `-=`: Subtracts the right operand from the left operand and assigns the result to the left operand.
- Multiplication assignment `*=`: Multiplies the left operand by the right operand and assigns the result to the left operand.
- Division assignment `/=`: Divides the left operand by the right operand and assigns the result to the left operand.
- Modulus assignment `%=`: Computes the modulus and assigns the result to the left operand.
- Exponentiation assignment `**=`: Computes the exponentiation and assigns the result to the left operand.
- Floor division assignment `//=`: Computes floor division and assigns the result to the left operand.

```
x = 10
y = 3

x += y # x is now 13
x -= y # x is now 10
x *= y # x is now 30
x /= y # x is now 10.0
x %= y # x is now 1.0
x **= y # x is now 1000.0
x //= y # x is now 333.0
```

Logical Operators

Logical operators perform logical operations on Boolean values.

- `and`: Returns `True` if both operands are `True`.
- `or`: Returns `True` if at least one operand is `True`.
- `not`: Returns the inverse of the operand.

```
has_access_card = True # The person has a valid access card
is_biometric_verified = False # However, the biometric verification failed
```

```
# Conditions for accessing a secure facility
both_conditions = has_access_card and is_biometric_verified
# Access allowed only if both conditions are true (having a valid access card and
passing biometric verification)

at_least_one_condition = has_access_card or is_biometric_verified
# Access allowed if at least one condition is true (either having a valid access
card or passing biometric verification)

inverse_condition = not is_biometric_verified
# Access allowed if the biometric verification fails (inverse of the biometric
verification result)
```

Bitwise Operators

Bitwise operators perform bit-level operations.

- `&` (Bitwise AND): Performs a bitwise AND operation.
- `|` (Bitwise OR): Performs a bitwise OR operation.
- `^` (Bitwise XOR): Performs a bitwise XOR (exclusive OR) operation.
- `~` (Bitwise NOT): Performs a bitwise NOT (complement) operation.
- `<<` (Left Shift): Shifts the bits to the left.
- `>>` (Right Shift): Shifts the bits to the right.

```
a = 5 # Binary: 0101
b = 3 # Binary: 0011

bitwise_and = a & b # Result: 1 (Binary: 0001)
bitwise_or = a | b # Result: 7 (Binary: 0111)
bitwise_xor = a ^ b # Result: 6 (Binary: 0110)
bitwise_not_a = ~a # Result: -6 (Binary: 11111010)
left_shift = a << 1 # Result: 10 (Binary: 1010)
right_shift = a >> 1 # Result: 2 (Binary: 0010)
```

These operators are essential for various programming tasks, including mathematical calculations, logical conditions, and low-level bit manipulation.

Topic 6: Decision Making

Decision Making in Python

Decision-making is a fundamental aspect of programming. Python provides constructs for making decisions using conditional statements.

`if` Statement

The `if` statement is used to execute a block of code if a condition is `True`.

```
age = 25

if age < 18:
    print("You are a minor.")
else:
    print("You are an adult.")
```

`elif` Statement

The `elif` statement is used for multiple conditional checks after an `if` statement.

```
grade = 85

if grade >= 90:
    print("A")
elif grade >= 80:
    print("B")
elif grade >= 70:
    print("C")
else:
    print("F")
```

`else` Statement

The `else` statement is used to execute a block of code if the condition in the `if` statement is `False`.

Topic 7: Looping in Python

While Loop

A **while** loop repeatedly executes a block of code as long as a specified condition is **True**.

```
count = 0

while count < 5:
    print(f"Count: {count}")
    count += 1
```

For Loop

A **for** loop is used to iterate over a sequence (e.g., a list, tuple, or string).

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(f"Current fruit: {fruit}")
```

Nested Loop

Nested loops are loops inside other loops. They are useful for iterating over complex data structures.

```
for i in range(3):
    for j in range(2):
        print(f"({i}, {j})")
```

Loop Control Statements

Loop control statements, such as **break** and **continue**, allow you to control the flow of loops.

break Statement

The **break** statement is used to exit a loop prematurely.

```
numbers = [1, 2, 3, 4, 5]
```

```
for number in numbers:
    if number == 3:
        break
    print(number)
```

`continue` Statement

The `continue` statement is used to skip the current iteration and continue to the next one.

```
numbers = [1, 2, 3, 4, 5]

for number in numbers:
    if number == 3:
        continue
    print(number)
```

Example: Using While and For Loops

```
# While loop
count = 0
while count < 5:
    print(f"Count (while): {count}")
    count += 1

# For loop
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(f"Current fruit (for): {fruit}")
```

In this example, we demonstrate the use of both `while` and `for` loops to achieve different types of iteration.

Topic 8: Conversion Functions (int(), long(), float())

Number Type Conversion

In Python, you can convert one numeric type to another using built-in functions. Common conversions include:

int()

The `int()` function converts a value to an integer.

```
x = int(5.6) # x will be 5
```

float()

The `float()` function converts a value to a floating-point number.

```
y = float("3.14") # y will be 3.14
```

Long Integers (Python 2)

In Python 2, there was a separate "long" integer type for very large integers. In Python 3, integers can represent very large numbers without the need for a separate "long" type.

```
# In Python 2 (not applicable in Python 3)
long_number = long(123456789012345678901234567890)
```

Example: Number Type Conversion

```
# Convert a float to an int
float_number = 3.7
int_number = int(float_number)
print(int_number) # Output: 3

# Convert a string to a float
str_float = "6.25"
float_from_str = float(str_float)
print(float_from_str) # Output: 6.25
```

In this example, we use the `int()` and `float()` functions to convert between numeric types.

Topic 9: Mathematical Operations and Functions

Mathematical Functions

Python provides a standard library of mathematical functions to perform various calculations. Some common mathematical functions include:

`abs()`

Returns the absolute value of a number.

```
x = abs(-5) # x will be 5
```

`pow()`

Returns the result of raising the first argument to the power of the second argument.

```
y = pow(2, 3) # y will be 8
```

`round()`

Rounds a floating-point number to the nearest integer.

```
z = round(3.67) # z will be 4
```

Random Function

The `random` module in Python allows you to work with random numbers.

`random()`

Returns a random floating-point number between 0 and 1.

```
import random

random_number = random.random()
```

`randint()`

Returns a random integer within a specified range.

```
import random
```

```
random_integer = random.randint(1, 10) # Generates a random integer between 1 and 10 (inclusive)
```

Trigonometric Functions

The `math` module in Python provides a range of trigonometric functions, including sine, cosine, and tangent.

```
import math

angle = math.pi / 4 # 45 degrees in radians

sin_value = math.sin(angle)
cos_value = math.cos(angle)
tan_value = math.tan(angle)
```

Example: Using Mathematical Functions and Random Numbers

```
import random
import math

# Absolute value
x = abs(-5) # x will be 5

# Power
y = pow(2, 3) # y will be 8

# Round
z = round(3.67) # z will be 4

# Random number between 0 and 1
random_number = random.random()

# Random integer between 1 and 10
random_integer = random.randint(1, 10)

# Trigonometric functions
angle = math.pi / 4
sin_value = math.sin(angle)
cos_value = math.cos(angle)
tan_value = math.tan(angle)
```

In this example, we use various mathematical functions and generate random numbers using the `random` module.

Topic 10: Strings - Escape Characters, String Special Operators, String Formatting Operators

Escape Characters

Escape characters in strings are used to insert special characters that are difficult to represent directly in a string.

- `\n`: Newline
- `\t`: Tab
- `\\`: Backslash
- `\"`: Double quote
- `\'`: Single quote

Example:

```
text = "Hello, \nWorld!"
```

String Special Operators

Python supports special operators for working with strings.

Concatenation (+)

Concatenation is used to join two or more strings together.

```
str1 = "Hello"  
str2 = "World"  
result = str1 + " " + str2 # "Hello World"
```

Repetition (*)

Repetition is used to repeat a string a specified number of times.

```
text = "Python"  
repeated_text = text * 3 # "PythonPythonPython"
```

String Formatting Operators

String formatting operators allow you to create formatted strings with placeholders for variable values.

- `%d`: Integer

- **%f**: Floating-point number
- **%s**: String

Example:

```
name = "Alice"
age = 30
height = 5.8
formatted_text = "Name: %s, Age: %d, Height: %.2f" % (name, age, height)
```

Example: Using Escape Characters and String Operators

```
# Escape characters
new_line = "Hello, \nWorld!"
tabbed_text = "This is a tabbed\ttext."

# String concatenation
str1 = "Hello"
str2 = "World"
concatenated_str = str1 + " " + str2 # "Hello World"

# String repetition
text = "Python"
repeated_text = text * 3 # "PythonPythonPython"

# String formatting
name = "Alice"
age = 30
height = 5.8
formatted_text = "Name: %s, Age: %d, Height: %.2f" % (name, age, height)
```

In this example, we use escape characters, string concatenation, repetition, and string formatting operators to work with strings.

Topic 11: Built-in String Methods

Python provides a wide range of built-in methods for manipulating strings.

center(width, fillchar)

The **center()** method returns a centered string, padded with a specified character to the given width.

```
text = "Python"
centered_text = text.center(10, "-") # "--Python--"
```

count(substring, start, end)

The `count()` method counts the number of occurrences of a substring in a string.

```
text = "Python is easy. Python is fun."  
count_py = text.count("Python") # 2
```

decode(encoding)

The `decode()` method decodes a string using a specified encoding.

```
encoded_text = b'Hello, World'  
decoded_text = encoded_text.decode("utf-8")
```

encode(encoding)

The `encode()` method encodes a string using a specified encoding.

```
text = "Hello, World"  
encoded_text = text.encode("utf-8")
```

Example: Using String Methods

```
# Center method  
text = "Python"  
centered_text = text.center(10, "-") # "--Python--"  
  
# Count method  
text = "Python is easy. Python is fun."  
count_py = text.count("Python") # 2  
  
# Decode and encode methods  
encoded_text = b'Hello, World'  
decoded_text = encoded_text.decode("utf-8")  
  
text = "Hello, World"  
encoded_text = text.encode("utf-8")
```

In this example, we use built-in string methods to center a string, count substrings, and encode/decode strings.

Topic 12: Python Lists - Working with Lists

Python Lists

Lists are ordered collections in Python, and they can hold elements of different data types. You can perform various operations on lists.

Accessing Values in Lists

You can access elements in a list using indexing.

```
fruits = ["apple", "banana", "cherry"]  
first_fruit = fruits[0] # "apple"  
second_fruit = fruits[1] # "banana"
```

Deleting List Elements

You can delete list elements using the `del` statement.

```
numbers = [1, 2, 3, 4, 5]  
del numbers[2] # Deletes the element at index 2
```

Indexing

List indexing starts at 0. You can also use negative indexing to access elements from the end of the list.

```
fruits = ["apple", "banana", "cherry"]  
last_fruit = fruits[-1] # "cherry"
```

Slicing

Slicing allows you to extract a portion of a list.

```
numbers = [1, 2, 3, 4, 5]  
subset = numbers[1:4] # [2, 3, 4]
```

Matrices

You can use lists to represent matrices.

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]  
value = matrix[1][2] # Accesses the element in the second row, third column (6)
```

Example: Working with Python Lists

```
# Accessing values in lists
fruits = ["apple", "banana", "cherry"]
first_fruit = fruits[0] # "apple"

# Deleting list elements
numbers = [1, 2, 3, 4, 5]
del numbers[2] # Removes the element at index 2

# Indexing and negative indexing
fruits = ["apple", "banana", "cherry"]
last_fruit = fruits[-1] # "cherry"

# Slicing
numbers = [1, 2, 3, 4, 5]
subset = numbers[1:4] # [2, 3, 4]

# Matrices
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
value = matrix[1][2] # 6
```

In this example, we demonstrate various operations you can perform with Python lists, including accessing values, deleting elements, indexing, slicing, and representing matrices.

Topic 13: Built-in Functions - cmp(), len(), min(), max(), list comprehension

Built-in Functions

Python provides several built-in functions that can be used with lists and other data structures.

len()

The `len()` function returns the number of items in a list or the length of a string.

```
fruits = ["apple", "banana", "cherry"]
length = len(fruits) # 3
```

min() and max()

The `min()` and `max()` functions return the minimum and maximum values in a list.

```
numbers = [5, 2, 8, 1, 9]
minimum = min(numbers) # 1
maximum = max(numbers) # 9
```

List Comprehension

List comprehension is a concise way to create lists.

```
squares = [x**2 for x in range(5)] # [0, 1, 4, 9, 16]

even_numbers_comprehension = [i for i in range(1, 11) if i % 2 == 0]

pairs_comprehension = [(x, y) for x in range(1, 4) for y in range(1, 3)]
```

Topic 14: Tuples - Accessing Values in Tuples, Deleting Tuple Elements, Indexing, Slicing, and Matrices

Tuples

Tuples are similar to lists but are immutable, which means you cannot change their elements once defined. Tuples are often used for data that should not be modified.

Accessing Values in Tuples

You can access elements in a tuple using indexing.

```
dimensions = (10, 20, 15)
width = dimensions[0] # 10
```

Deleting Tuple Elements

You cannot delete individual elements from a tuple, but you can delete the entire tuple.

```
dimensions = (10, 20, 15)
del dimensions # Deletes the entire tuple
```

Indexing

Tuple indexing works similarly to list indexing.

```
fruits = ("apple", "banana", "cherry")
last_fruit = fruits[-1] # "cherry"
```

Slicing

Slicing is also applicable to tuples.

```
numbers = (1, 2, 3, 4, 5)
subset = numbers[1:4] # (2, 3, 4)
```

Matrices

You can use tuples to represent matrices just like lists.

```
matrix = ((1, 2, 3), (4, 5, 6), (7, 8, 9))
value = matrix[1][2] # Accesses the element in the second row, third column (6)
```

Example: Working with Tuples

```
# Accessing values in tuples
dimensions = (10, 20, 15)
width = dimensions[0] # 10

# Deleting a tuple
fruits = ("apple", "banana", "cherry")
del fruits # Deletes the entire tuple

# Indexing and negative indexing
fruits = ("apple", "banana", "cherry")
last_fruit = fruits[-1] # "cherry"

# Slicing
numbers = (1, 2, 3, 4, 5)
subset = numbers[1:4] # (2, 3, 4)

# Matrices with tuples
matrix = ((1, 2, 3), (4, 5, 6), (7, 8, 9))
value = matrix[1][2] # 6
```

In this example, we explore the use of tuples, including accessing values, deleting tuples, indexing, slicing, and representing matrices using tuples.

Topic 15: Built-in Tuple Functions - cmp(), len()

Built-in Tuple Functions

Python provides built-in functions that can be used with tuples.

`len()`

The `len()` function returns the number of items in a tuple.

```
fruits = ("apple", "banana", "cherry")
length = len(fruits) # 3
```

Example: Using Built-in Tuple Functions

```
# Length of a tuple
fruits = ("apple", "banana", "cherry")
length = len(fruits) # 3
```

In this example, we use the `len()` function to obtain the length of a tuple.

Topic 16: Dictionaries - Accessing Values from Dictionaries, Deleting and Updating Elements in Dictionaries

Dictionaries in Python

Dictionaries are collections of key-value pairs. Each key is unique and maps to a specific value.

Accessing Values from Dictionaries

You can access values in a dictionary by using the key.

```
person = {"name": "John", "age": 30}
person_name = person["name"] # "John"
```

Deleting and Updating Elements in Dictionaries

You can delete key-value pairs or update values in dictionaries.

```
# Deleting a key-value pair
del person["age"]
```

```
# Updating a value
person["city"] = "New York"
```

Example: Working with Dictionaries

```
# Accessing values from dictionaries
person = {"name": "John", "age": 30}
person_name = person["name"] # "John"

# Deleting a key-value pair
del person["age"]

# Updating a value
person["city"] = "New York"
```

In this example, we explore the use of dictionaries, including accessing values, deleting key-value pairs, and updating values in dictionaries.

Topic 17: Properties of Dictionaries, Built-in Dictionary Functions & Methods, Dictionary Comprehension

Properties of Dictionaries

Dictionaries in Python have the following properties:

- Keys are unique.
- Keys are case-sensitive.
- Values can be of any data type.
- Dictionaries are unordered (in Python 3.7+ and CPython 3.6, dictionaries are ordered).

Built-in Dictionary Functions & Methods

Python provides built-in functions and methods to work with dictionaries.

len()

The **len()** function returns the number of key-value pairs in a dictionary.

```
person = {"name": "John", "age": 30}
```

```
length = len(person) # 2
```

keys()

The `keys()` method returns a list of all keys in a dictionary.

```
person = {"name": "John", "age": 30}
key_list = person.keys() # ["name", "age"]
```

values()

The `values()` method returns a list of all values in a dictionary.

```
person = {"name": "John", "age": 30}
value_list = person.values() # ["John", 30]
```

items()

The `items()` method returns a list of key-value pairs (tuples) in a dictionary.

```
person = {"name": "John", "age": 30}
item_list = person.items() # [("name", "John"), ("age", 30)]
```

Dictionary Comprehension

Dictionary comprehension allows you to create dictionaries in a concise way.

```
squares = {x: x**2 for x in range(5)} # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Example: Working with Dictionaries and Built-in Functions

```
# Properties of dictionaries
person = {"name": "John", "age": 30}

# Length of a dictionary
length = len(person) # 2

# Dictionary keys
key_list = person.keys() # ["name", "age"]

# Dictionary values
```

```
value_list = person.values() # ["John", 30]

# Dictionary items
item_list = person.items() # [("name", "John"), ("age", 30)]

# Dictionary comprehension
squares = {x: x**2 for x in range(5)} # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

In this example, we explore the properties of dictionaries, use built-in functions and methods to work with dictionaries, and create dictionaries using comprehension.

Topic 18: Date & Time - Time Tuple, Calendar Module, and Time Module

Date & Time in Python

Python provides modules for working with date and time, allowing you to manipulate and display dates and times.

Time Tuple

A time tuple is a 9-tuple that represents time. The elements of the time tuple are year, month, day, hour, minute, second, day of the week, day of the year, and daylight saving time.

```
import time

current_time = time.localtime()
year = current_time.tm_year
month = current_time.tm_mon
```

Calendar Module

The **calendar** module provides functions to work with calendars.

```
import calendar

cal = calendar.month(2023, 11)
print(cal)
```

Time Module

The `time` module provides functions for working with time, including sleeping, measuring time, and formatting time.

```
import time

current_time = time.ctime()
print(current_time)

# Sleep for 2 seconds
time.sleep(2)
```

Example: Working with Date & Time

```
import time
import calendar

# Time tuple
current_time = time.localtime()
year = current_time.tm_year
month = current_time.tm_mon

# Calendar module
cal = calendar.month(2023, 11)
print(cal)

# Time module
current_time = time.ctime()
print(current_time)

# Sleep for 2 seconds
time.sleep(2)
```

In this example, we use the `time` and `calendar` modules to work with date and time. We obtain the current time, display a calendar for a specific month, and sleep for 2 seconds using the `time.sleep()` function.

Topic 19: Functions - Define Function, Calling Function, Pass by Reference as Value, Function Arguments, Anonymous Functions, Return Statements

Functions in Python

Functions are reusable blocks of code that perform specific tasks. In Python, you can define your own functions and use built-in functions.

Defining a Function

To define a function, use the `def` keyword followed by the function name and parameters.

```
def greet(name):  
    print(f"Hello, {name}!")
```

Calling a Function

To call a function, use its name followed by parentheses and any required arguments.

```
greet("Alice")
```

Pass by Object Reference (or Call by Object Reference)

```
# Function that modifies a list  
def modify_list(my_list):  
    print("Inside function (before modification):", my_list)  
    my_list.append(4)  
    print("Inside function (after modification):", my_list)  
  
# Main code  
original_list = [1, 2, 3]  
print("Before function call:", original_list)  
modify_list(original_list)  
print("After function call:", original_list)
```

In this example, the function `modify_list` takes a list as an argument. The list is mutable, so any modifications made to it inside the function affect the original list outside the function. This behavior is consistent with the idea of "pass by object reference" in Python.

```
# Function that reassigns the parameter
def reassign_value(x):
    print("Inside function (before reassignment):", x)
    x = 10
    print("Inside function (after reassignment):", x)

# Main code
original_value = 5
print("Before function call:", original_value)
reassign_value(original_value)
print("After function call:", original_value)
```

Pass by Value (or Pass by Assignment)

```
# Function that reassigns the parameter
def reassign_value(x):
    print("Inside function (before reassignment):", x)
    x = 10
    print("Inside function (after reassignment):", x)

# Main code
original_value = 5
print("Before function call:", original_value)
reassign_value(original_value)
print("After function call:", original_value)
```

In this example, the function `reassign_value` takes an integer as an argument. However, when the value is reassigned inside the function, it does not affect the original value outside the function. This is because integers in Python are immutable, and when reassigned, a new reference is created.

Pass by Reference as Value

In Python, function arguments are passed by reference as value. This means that when you pass an argument to a function, a copy of the reference to the object is passed. This allows the function to modify the object, but it won't affect the original object outside the function.

Function Arguments

Functions can have positional arguments and keyword arguments.

```
def calculate(a, b=10):  
    return a + b  
  
result1 = calculate(5) # result1 is 15 (default b=10)  
result2 = calculate(5, 7) # result2 is 12
```

Anonymous Functions (Lambda Functions)

Lambda functions are small, anonymous functions defined using the `lambda` keyword.

```
multiply = lambda x, y: x * y  
result = multiply(3, 4) # result is 12
```

Return Statements

Functions can return values using the `return` statement.

```
def add(a, b):  
    return a + b
```

Example: Defining, Calling, and Using Functions

```
# Defining a function  
def greet(name):  
    print(f"Hello, {name}!")  
  
# Calling a function  
greet("Alice")  
  
# Pass by Reference as Value  
def modify_list(my_list):  
    my_list.append(42)  
  
original_list = [1, 2, 3]  
modify_list(original_list)  
print(original_list) # [1, 2, 3, 42]  
  
# Function arguments  
def calculate(a, b=10):  
    return a + b  
  
result1 = calculate(5) # result1 is 15 (default b=10)
```

```

result2 = calculate(5, 7) # result2 is 12

# Anonymous function (Lambda)
multiply = lambda x, y: x * y
result = multiply(3, 4) # result is 12

# Return statement
def add(a, b):
    return a + b

result = add(3, 5) # result is 8

```

In this example, we define functions, call them, demonstrate pass by reference as value, work with function arguments, use an anonymous function, and return values from functions.

Topic 20: Scope of Variables - Local & Global, Decorators, and Recursion

Scope of Variables

In Python, variables have different scopes:

- Local variables are defined within a function and are only accessible within that function.
- Global variables are defined outside of functions and can be accessed from anywhere in the code.

```

global_var = 10 # Global variable

def my_function():
    local_var = 5 # Local variable
    print(local_var) # Accessible
    print(global_var) # Accessible

my_function()
print(local_var) # Not accessible
print(global_var) # Accessible

```

Decorators

Decorators are a powerful and flexible way to modify or enhance the behavior of functions or methods. They are often used for tasks like logging, authentication, and more.

```
def my_decorator(func):
    def wrapper():
        print("Something is happening before the function is called.")
        func()
        print("Something is happening after the function is called.")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
```

Recursion

Recursion is a technique in which a function calls itself to solve a problem. It's particularly useful for solving problems that can be broken down into smaller, similar subproblems.

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)
result = factorial(5) # result is 120
```

Topic 21: Map, Reduce, and Filter

Map, Reduce, and Filter

`map()`, `reduce()`, and `filter()` are built-in functions in Python that are used for performing operations on data.

`map()`

The `map()` function applies a given function to each item of an iterable (e.g., a list) and returns a new iterable with the results.

```
def square(x):
    return x**2
```

```
numbers = [1, 2, 3, 4, 5]
squared_numbers = list(map(square, numbers))
# squared_numbers is [1, 4, 9, 16, 25]
```

reduce()

The `reduce()` function is used to apply a function of two arguments cumulatively to the items of an iterable, reducing it to a single value.

```
from functools import reduce

def add(x, y):
    return x + y

numbers = [1, 2, 3, 4, 5]
result = reduce(add, numbers)
# result is 15
```

filter()

The `filter()` function filters elements of an iterable based on a given function and returns an iterable with the elements for which the function returns `True`.

```
def is_even(x):
    return x % 2 == 0

numbers = [1, 2, 3, 4, 5]
even_numbers = list(filter(is_even, numbers))
# even_numbers is [2, 4]
```

Topic 22: Import Statements, Locating Modules - Current Directory, Pythonpath

Import Statements

In Python, you can import modules or packages to extend the functionality of your code. Import statements allow you to use functions, classes, and variables defined in other Python files.

Importing a Module

You can import an entire module using the `import` statement. For example, to import the `math` module:

```
import math
```

You can then access functions and variables from the `math` module using dot notation, like `math.sqrt()`.

Importing Specific Items

You can import specific functions or variables from a module using the `from` keyword. For example, to import only the `sqrt` function from the `math` module:

```
from math import sqrt
```

Using Aliases

You can use aliases to make module names shorter and more readable. For example, to import the `numpy` module as `np`:

```
import numpy as np
```

Locating Modules - Current Directory, Pythonpath

When you import a module, Python searches for it in several places, including the current directory and directories listed in the `sys.path` list. The order in which Python searches for modules is as follows:

1. The current directory.
2. Directories specified in the `PYTHONPATH` environment variable.
3. Standard library directories.

You can add directories to the `PYTHONPATH` to extend the search path for Python modules.

Example: Importing Modules and Modifying the Search Path

```
# Importing modules
import math
from math import sqrt
import numpy as np
```

```
# Using the imported modules
value = math.sqrt(25) # 5.0
value2 = sqrt(16) # 4.0
array = np.array([1, 2, 3])

# Modifying the search path
import sys
sys.path.append("/path/to/your/directory")
```

In this example, we demonstrate how to import modules, access functions and variables from them, and how to modify the search path for Python modules.

Topic 23: Dir() Function, Global and Location Functions, Reload() Function, Sys Module, and Subprocess Module

Dir() Function

The `dir()` function is used to list all the names in the current scope or the attributes of an object. It returns a list of names.

```
import math
attributes = dir(math)
```

Global and Local Functions

In Python, you can determine whether a variable is in the global or local scope using the `globals()` and `locals()` functions.

```
x = 10 # Global variable

def my_function():
    y = 5 # Local variable
    local_variables = locals()
    global_variables = globals()
```

Reload() Function

The `reload()` function is used to reload a module that has been previously imported. It can be helpful when you're testing and modifying a module without restarting the interpreter.

```
import mymodule
# Modify mymodule.py
import importlib
importlib.reload(mymodule)
```

Sys Module

The **sys** module provides access to some variables used or maintained by the interpreter and functions that interact with the interpreter.

```
import sys
version_info = sys.version_info
```

Subprocess Module

The **subprocess** module allows you to spawn new processes, connect to their input/output/error pipes, and obtain their return codes.

```
import subprocess
result = subprocess.run(["ls", "-l"], stdout=subprocess.PIPE, text=True)
```

Example: Using Dir() Function, Global and Local Functions, Reload() Function, Sys Module, and Subprocess Module

```
# Dir() function
import math
attributes = dir(math)

# Global and local functions
x = 10 # Global variable

def my_function():
    y = 5 # Local variable
    local_variables = locals()
    global_variables = globals()

# Reload() function
import mymodule # Assume mymodule.py is modified
import importlib
```

```
importlib.reload(mymodule)

# Sys module
import sys
version_info = sys.version_info

# Subprocess module
import subprocess
result = subprocess.run(["ls", "-l"], stdout=subprocess.PIPE, text=True)
```

In this example, we use the `dir()` function to list attributes, check global and local variables using `globals()` and `locals()`, reload a modified module, access information from the `sys` module, and run a subprocess using the `subprocess` module.

Topic 24: Packages in Python

Packages

In Python, a package is a collection of modules organized into directories. Packages allow you to structure your code into a hierarchy, making it more organized and manageable.

Creating a Package

To create a package, you need to organize your modules into directories. A package directory should contain an `__init__.py` file, which can be empty or contain package-level initialization code.

```
my_package/
  __init__.py
  module1.py
  module2.py
```

Importing from a Package

You can import modules and submodules from a package using dot notation.

```
import my_package.module1
from my_package.module2 import some_function
```

Relative Imports

You can use relative imports to import modules within the same package without specifying the full package path.

```
from .module1 import my_function
```

Example: Creating and Using a Package

```
my_package/  
    __init__.py  
    module1.py  
    module2.py
```

```
# Inside module1.py  
def my_function():  
    print("Function in module1")  
  
# Inside module2.py  
def some_function():  
    print("Function in module2")
```

```
# Using the package and its modules  
import my_package.module1  
my_package.module1.my_function()  
  
from my_package.module2 import some_function  
some_function()
```

In this example, we create a simple package named `my_package` with two modules. We demonstrate how to import and use functions from these modules.

Topic 25: Files in Python - Reading Keyboard Input, Input Function, Opening and Closing Files, Syntax, and List of Modes

Files in Python

Working with files is a common task in Python. You can read, write, and manipulate files to store and retrieve data.

Reading Keyboard Input

You can use the `input()` function to read input from the user. It reads a line of text from the user and returns it as a string.

```
user_input = input("Enter your name: ")
```

Opening and Closing Files

You can use the `open()` function to open a file and the `close()` method to close it. It's important to close files when you're done to free up system resources.

```
file = open("example.txt", "r")
content = file.read()
file.close()
```

Syntax for Opening Files

The `open()` function takes two arguments: the file name and the mode in which you want to open the file. Common modes include:

- "r" for reading (default).
- "w" for writing.
- "a" for appending.
- "b" for binary mode.
- "t" for text mode (default).

Example: Reading Keyboard Input and Opening/Closing Files

```
# Reading keyboard input
user_input = input("Enter your name: ")
print(f"Hello, {user_input}!")

# Opening and closing files
file = open("example.txt", "r")
content = file.read()
file.close()
```

In this example, we use `input()` to read user input and the `open()` and `close()` functions to work with files.

Topic 26: Files Object Attributes - open, close, Reading and Writing Files, File Position

File Object Attributes

When you open a file in Python, you get a file object that has various attributes and methods for reading and writing.

Opening Files

You can open a file using the `open()` function, which returns a file object.

```
file = open("example.txt", "r")
```

Closing Files

Always remember to close files using the `close()` method when you're done with them to free up system resources.

```
file.close()
```

Reading Files

You can read the content of a file using methods like `read()`, `readline()`, or `readlines()`.

```
content = file.read()
line = file.readline()
lines = file.readlines()
```

Writing to Files

To write to a file, open it in write mode ("w") or append mode ("a") and use the `write()` method.

```
file = open("output.txt", "w")
file.write("This is a sample line.")
file.close()
```

File Position

You can move the file cursor to a specific position using the `seek()` method and check the current position with `tell()`.

```
file.seek(0) # Move to the beginning
position = file.tell() # Get the current position
```

Example: Working with File Object Attributes

```
# Opening and closing files
file = open("example.txt", "r")
file.close()

# Reading files
file = open("example.txt", "r")
content = file.read()
line = file.readline()
lines = file.readlines()
file.close()

# Writing to files
file = open("output.txt", "w")
file.write("This is a sample line.")
file.close()

# File position
file = open("example.txt", "r")
file.seek(10) # Move to position 10
position = file.tell() # Get the current position
```

In this example, we explore the various attributes and methods associated with file objects, including opening, closing, reading, writing, and manipulating the file position.

Topic 27: Renaming and Deleting Files

Renaming Files

You can use the `os` module to rename files. The `os.rename()` function allows you to change the name of a file.

```
import os

old_name = "old_file.txt"
new_name = "new_file.txt"
```

```
os.rename(old_name, new_name)
```

Deleting Files

To delete a file, you can use the `os.remove()` function.

```
import os

file_to_delete = "file_to_delete.txt"
os.remove(file_to_delete)
```

Example: Renaming and Deleting Files

```
import os

# Renaming a file
old_name = "old_file.txt"
new_name = "new_file.txt"
os.rename(old_name, new_name)

# Deleting a file
file_to_delete = "file_to_delete.txt"
os.remove(file_to_delete)
```

In this example, we use the `os` module to rename and delete files.

Topic 28: Pickle and JSON

Pickle

Pickle is a Python module used to serialize and deserialize Python objects. It allows you to save complex data structures to a file and load them back into memory when needed.

Pickling an Object

You can use the `pickle.dump()` function to serialize an object and save it to a file.

```
import pickle

data = [1, 2, 3, 4, 5]
with open("data.pkl", "wb") as file:
    pickle.dump(data, file)
```

Unpickling an Object

To read a pickled object from a file and deserialize it, you can use the `pickle.load()` function.

```
import pickle

with open("data.pkl", "rb") as file:
    loaded_data = pickle.load(file)
```

JSON (JavaScript Object Notation)

JSON is a lightweight data interchange format that is easy for humans to read and write. Python has a built-in `json` module for working with JSON data.

Encoding to JSON

You can use the `json.dump()` function to convert a Python object to a JSON string or save it to a file.

```
import json

data = {"name": "Alice", "age": 30}
json_data = json.dumps(data)
```

Decoding JSON

To decode JSON data, use the `json.loads()` function for parsing a JSON string, or `json.load()` to read from a file.

```
import json

json_data = '{"name": "Alice", "age": 30}'
```

```
data = json.loads(json_data)
```

Example: Pickle and JSON

```
import pickle
import json

# Pickling an object
data = [1, 2, 3, 4, 5]
with open("data.pkl", "wb") as file:
    pickle.dump(data, file)

# Unpickling an object
with open("data.pkl", "rb") as file:
    loaded_data = pickle.load(file)

# Encoding to JSON
data = {"name": "Alice", "age": 30}
json_data = json.dumps(data)

# Decoding JSON
json_data = '{"name": "Alice", "age": 30}'
data = json.loads(json_data)
```

In this example, we use the `pickle` module to pickle and unpickle objects and the `json` module to encode and decode JSON data.

Topic 29: mkdir Method, chdir() Method, getcwd Method, rmdir

Working with Directories

Python provides functions to create, change, and manage directories.

Creating a Directory

You can use the `os.mkdir()` function to create a new directory.

```
import os

directory_name = "my_directory"
os.mkdir(directory_name)
```

Changing the Current Directory

You can use the `os.chdir()` function to change the current working directory.

```
import os

new_directory = "/path/to/new/directory"
os.chdir(new_directory)
```

Getting the Current Working Directory

The `os.getcwd()` function returns the current working directory as a string.

```
import os

current_directory = os.getcwd()
```

Removing a Directory

You can use the `os.rmdir()` function to remove an empty directory.

```
import os

directory_to_remove = "directory_to_remove"
os.rmdir(directory_to_remove)
```

Example: Working with Directories

```
import os

# Creating a directory
```

```
directory_name = "my_directory"
os.mkdir(directory_name)

# Changing the current directory
new_directory = "/path/to/new/directory"
os.chdir(new_directory)

# Getting the current working directory
current_directory = os.getcwd()

# Removing a directory
directory_to_remove = "directory_to_remove"
os.rmdir(directory_to_remove)
```

In this example, we demonstrate how to create, change, and manage directories using functions from the `os` module.

Topic 30: Exception Handling - List of Exceptions, Try and Except

Exception Handling

Exception handling allows you to handle errors or exceptions gracefully in your code, preventing it from crashing.

List of Exceptions

Python has a variety of built-in exceptions, such as `ZeroDivisionError`, `TypeError`, and `FileNotFoundError`. You can find a list of built-in exceptions in the Python documentation.

Try and Except

The `try` and `except` blocks are used for exception handling. Code inside the `try` block is executed, and if an exception occurs, it's caught by the `except` block.

```
try:
    result = 10 / 0
except ZeroDivisionError:
    result = "Division by zero is not allowed"
```

You can also catch multiple exceptions or use a generic `except` block.

```
try:
    result = 10 / 0
except (ZeroDivisionError, TypeError) as e:
    result = f"An error occurred: {e}"
except Exception as e:
    result = f"A generic error occurred: {e}"
```

Example: Exception Handling

```
# Handling a specific exception
try:
    result = 10 / 0
except ZeroDivisionError:
    result = "Division by zero is not allowed"

# Handling multiple exceptions
try:
    result = 10 / "string"
except (ZeroDivisionError, TypeError) as e:
    result = f"An error occurred: {e}"

# Handling a generic exception
try:
    result = 10 / 0
except Exception as e:
    result = f"A generic error occurred: {e}"
```

In this example, we demonstrate exception handling using the `try` and `except` blocks, catching specific exceptions, multiple exceptions, and a generic exception.

Topic 31: Try-Finally Clause and User-Defined Exceptions

Try-Finally Clause

The `try` and `finally` blocks allow you to ensure that certain code is executed, regardless of whether an exception is raised or not. The code in the `finally` block will always run.

```
try:
    # Some code that may raise an exception
    result = 10 / 0
finally:
    # This code always runs
    print("Execution completed.")
```

User-Defined Exceptions

You can create your own custom exceptions by creating a new class that inherits from the `Exception` class or one of its subclasses.

```
class MyCustomException(Exception):
    def __init__(self, message):
        self.message = message

try:
    raise MyCustomException("This is a custom exception.")
except MyCustomException as e:
    print(f"Caught my custom exception: {e.message}")
```

Example: Try-Finally Clause and User-Defined Exceptions

```
# Try-Finally Clause
try:
    result = 10 / 0
finally:
    print("Execution completed.")

# User-Defined Exception
class MyCustomException(Exception):
    def __init__(self, message):
        self.message = message

try:
    raise MyCustomException("This is a custom exception.")
except MyCustomException as e:
    print(f"Caught my custom exception: {e.message}")
```

In this example, we show how to use the `try` and `finally` blocks to ensure execution of specific code and how to create and catch user-defined exceptions.

Topic 32: Object-Oriented Programming (OOP) Concepts, Class, Objects, Inheritance, Overriding Methods, Overloading Operators, Data Hiding

Object-Oriented Programming (OOP) Concepts

Object-Oriented Programming is a programming paradigm that uses objects to structure code. In Python, everything is an object, and you can create your own custom classes and objects.

Class

A class is a blueprint for creating objects. It defines the attributes (variables) and methods (functions) that the objects will have.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        return f"Hello, my name is {self.name} and I'm {self.age} years old."
```

Objects

Objects are instances of classes. You can create multiple objects from a single class.

```
person1 = Person("Alice", 30)
person2 = Person("Bob", 25)
```

Inheritance

Inheritance allows a new class (subclass or derived class) to inherit properties and methods from an existing class (base class or superclass). It promotes code reuse.

```
class Student(Person):
    def __init__(self, name, age, student_id):
        super().__init__(name, age)
        self.student_id = student_id
```

```
def study(self):  
    return f"{self.name} is studying."
```

Overriding Methods

Subclasses can override methods from their superclass to provide their own implementation.

```
class Student(Person):  
    def greet(self):  
        return f"Hello, I'm {self.name}, a student."  
  
student = Student("Charlie", 18, "12345")  
print(student.greet()) # Output: "Hello, I'm Charlie, a student."
```

Overloading Operators

Python allows operator overloading, which means you can define how operators behave for your custom objects by implementing special methods like `__add__`, `__sub__`, and others.

```
class Point:  
    def __init__(self, x, y):  
        self.x = x  
        self.y = y  
  
    def __add__(self, other):  
        return Point(self.x + other.x, self.y + other.y)  
  
point1 = Point(1, 2)  
point2 = Point(3, 4)  
result = point1 + point2 # Calls the __add__ method
```

Data Hiding

In Python, you can use name mangling to make instance variables or methods private by adding a double underscore prefix.

```
class MyClass:  
    def __init__(self):  
        self.__my_private_var = 42
```

```
def get_private_var(self):  
    return self.__my_private_var
```

Example: Object-Oriented Programming in Python

```
class Person:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
  
    def greet(self):  
        return f"Hello, my name is {self.name} and I'm {self.age} years old."  
  
class Student(Person):  
    def __init__(self, name, age, student_id):  
        super().__init__(name, age)  
        self.student_id = student_id  
  
    def study(self):  
        return f"{self.name} is studying."  
  
class Point:  
    def __init__(self, x, y):  
        self.x = x  
        self.y = y  
  
    def __add__(self, other):  
        return Point(self.x + other.x, self.y + other.y)  
  
point1 = Point(1, 2)  
point2 = Point(3, 4)  
result = point1 + point2  
  
class MyClass:  
    def __init__(self):  
        self.__my_private_var = 42  
  
    def get_private_var(self):  
        return self.__my_private_var
```

In this example, we explore various OOP concepts, including classes, objects, inheritance, method overriding, operator overloading, and data hiding.

Topic 33: match Function, search Function, Matching vs. Searching

match Function

The `match()` function in Python is used for regular expression pattern matching at the beginning of a string. It returns a match object if the pattern is found at the start of the string, or `None` if there is no match.

```
import re

pattern = r"Hello"
text = "Hello, World!"

match = re.match(pattern, text)
if match:
    print("Pattern found:", match.group())
else:
    print("Pattern not found")
```

search Function

The `search()` function is used for regular expression pattern matching anywhere in a string. It searches for the pattern in the entire string and returns a match object if a match is found, or `None` if there is no match.

```
import re

pattern = r"World"
text = "Hello, World!"

search = re.search(pattern, text)
if search:
    print("Pattern found:", search.group())
else:
    print("Pattern not found")
```

Matching vs. Searching

- `match()` checks if the pattern is at the beginning of the string, so it's useful for validating or extracting data from the start of a string.
- `search()` looks for the pattern anywhere in the string, so it's suitable for finding occurrences of a pattern within the text.

Example: Using `match()` and `search()`

```
import re

# Using match()
pattern = r"Hello"
text = "Hello, World!"
match = re.match(pattern, text)
if match:
    print("Pattern found with match:", match.group())
else:
    print("Pattern not found with match")

# Using search()
pattern = r"World"
text = "Hello, World!"
search = re.search(pattern, text)
if search:
    print("Pattern found with search:", search.group())
else:
    print("Pattern not found with search")
```

In this example, we demonstrate the use of the `match()` and `search()` functions for regular expression pattern matching in Python.

Topic 35: Database Connectivity - Create, Insert, Update, and Delete Operations, Handling Errors

Database Connectivity

Python allows you to connect to databases using various libraries like SQLite, MySQL, or PostgreSQL. In this example, we'll use SQLite as a simple database for demonstration.

Connecting to a Database

To connect to a SQLite database, you can use the `sqlite3` library.

```
import sqlite3

conn = sqlite3.connect("mydatabase.db")
```

Creating a Table

You can create a table in the database to store data.

```
cursor = conn.cursor()
cursor.execute('''CREATE TABLE IF NOT EXISTS employees (id INTEGER PRIMARY KEY, name TEXT, salary REAL)''')
```

Inserting Data

You can insert data into the database.

```
cursor.execute("INSERT INTO employees (name, salary) VALUES (?, ?)", ("Alice", 50000))
conn.commit()
```

Updating Data

You can update existing data in the database.

```
cursor.execute("UPDATE employees SET salary = 55000 WHERE name = 'Alice'")
conn.commit()
```

Deleting Data

You can delete data from the database.

```
cursor.execute("DELETE FROM employees WHERE name = 'Alice'")
conn.commit()
```

Handling Errors

When working with databases, it's important to handle errors, especially when executing SQL commands.

```
try:
    cursor.execute("INSERT INTO employees (name, salary) VALUES (?, ?)", ("Bob",
    "invalid_salary"))
    conn.commit()
except sqlite3.Error as e:
    print("An error occurred:", e)
    conn.rollback()
```

Example: Database Connectivity and Operations

```
import sqlite3

#Connecting to a database
conn = sqlite3.connect("mydatabase.db")

#Creating a table
cursor = conn.cursor()
cursor.execute('''CREATE TABLE IF NOT EXISTS employees (id INTEGER PRIMARY KEY, name
TEXT, salary REAL)''')

#Inserting data
cursor.execute("INSERT INTO employees (name, salary) VALUES (?, ?)", ("Alice",
50000))
conn.commit()

#Fetching data
cursor.execute("SELECT * FROM employees")
data = cursor.fetchall()
for row in data:
    print("ID: ", row[0])
```

```

print("Name: ", row[1])
print("Salary: ", row[2])

#Updating data
cursor.execute("UPDATE employees SET salary = 55000 WHERE name = 'Alice'")
conn.commit()

#Deleting data
cursor.execute("DELETE FROM employees WHERE name = 'Alice'")
conn.commit()

#Handling errors
try:
    cursor.execute("INSERT INTO employees (name, salary) VALUES (?, ?)", ("Bob",
"invalid_salary"))
    conn.commit()
except sqlite3.Error as e:
    print("An error occurred:", e)
    conn.rollback()

```

In this example, we demonstrate how to connect to a SQLite database, create a table, perform insert, update, and delete operations, and handle errors when working with databases.

Topic 34: Regular Expression Modifiers and Patterns

Regular expressions (regex or regexp) are a powerful tool for pattern matching and text manipulation in Python. They are used to search, match, and manipulate strings based on specific patterns.

Regular Expression Modifiers

Modifiers are used to change the meaning of a regular expression. In Python, you can use the following modifiers:

- `re.I` or `re.IGNORECASE`: Case-insensitive matching.

- `re.M` or `re.MULTILINE`: Match the start and end of each line (not just the start and end of the whole string).
- `re.S` or `re.DOTALL`: Dot matches all, including newlines.
- `re.X` or `re.VERBOSE`: Allows you to write regular expressions more readable by ignoring whitespace and comments.

Regular Expression Patterns

Patterns are the expressions that specify what you're looking for in a string. Here are some common patterns and special characters in regular expressions:

- `.` (dot): Matches any character except a newline.
- `*`: Matches zero or more occurrences of the preceding character or group.
- `+`: Matches one or more occurrences of the preceding character or group.
- `?`: Matches zero or one occurrence of the preceding character or group.
- `|`: Acts like a logical OR.
- `[]`: Indicates a character class, matching any character within the brackets.
- `[^]`: Indicates a negated character class, matching any character not within the brackets.
- `()` (parentheses): Groups characters, allowing for complex expressions.
- `^`: Matches the start of a string.
- `$`: Matches the end of a string.

Example: Using Regular Expression Modifiers and Patterns

```
import re

text = "Hello, my name is Alice. My email is alice@example.com. My friend's email is Bob@example.com."

# Using re.IGNORECASE modifier
pattern = r"alice"
match = re.search(pattern, text, re.IGNORECASE)
if match:
    print(f"Match found: {match.group()}")

# Using re.MULTILINE modifier
pattern = r"^My email"
matches = re.finditer(pattern, text, re.MULTILINE)
for match in matches:
```

```

    print(f"Match found: {match.group()}")

# Using character classes and groups
pattern = r"[A-Za-z]+@[A-Za-z]+\.[A-Za-z]+"
emails = re.findall(pattern, text)
for email in emails:
    print(f"Email found: {email}")

```

In this example, we use regular expression modifiers and patterns to search and extract information from a text string, including case-insensitive matching, matching at the start of lines, and extracting email addresses.

Examples of modifiers

1. Case-Insensitive Matching (re.I or re.IGNORECASE):

- `re.search(r'apple', text, re.I)` matches "apple," "Apple," "APPLE," etc., in the text.

2. Multiline Matching (re.M or re.MULTILINE):

- `re.search(r'^start', text, re.M)` matches "start" at the beginning of each line in the text.

3. Dot Matches All (re.S or re.DOTALL):

- `re.search(r'.+', text, re.S)` matches any character, including newlines.

4. Verbose Mode (re.X or re.VERBOSE):

- You can write complex regular expressions more readable with whitespace and comments:

```

pattern = re.compile(r'''
    (\d{4})    # Match a 4-digit year
    -         # Match a hyphen
    (\d{2})    # Match a 2-digit month
    -         # Match a hyphen
    (\d{2})    # Match a 2-digit day
''', re.X)

```

These modifiers allow you to change the behavior of regular expressions to suit your specific needs when searching and matching patterns in text.

Topics explained

1. Python Introduction - Python follows a programming cycle involving writing, interpreting, and executing code, making it a versatile and easy-to-learn language.
2. Python IDE - Python Integrated Development Environments (IDEs) are software tools that provide a convenient environment for writing, testing, and debugging Python code.
3. Variables, Data Types - Python supports variables and various data types such as integers, floats, strings, and booleans for flexible data manipulation.
4. Numbers, Strings, List, Tuple, Dictionary - Python offers data structures like numbers, strings, lists, tuples, and dictionaries for organizing and manipulating data.
5. Operators - Python supports arithmetic, comparison, assignment, logical, and bitwise operators to perform various operations on data.
6. Decision Making - Loops - Python allows decision making and looping constructs for control flow in programs.
7. While Loop, For Loop, and Nested Loop - Python provides while loops, for loops, and the ability to nest loops for iterative tasks.
8. Number Type Conversion - Python includes functions like `int()`, `long()`, and `float()` for converting between different numeric types.
9. Mathematical Functions, Random Function, Trigonometric Function - Python includes functions for mathematical operations, random number generation, and trigonometric calculations.
10. Strings - Escape Characters, String Special Operators, String Formatting Operators - Python offers features like escape characters and string operators for working with text.
11. Built-in String Methods - Python provides built-in string methods like `center()`, `count()`, `decode()`, and `encode()` for text manipulation.
12. Python Lists - Lists in Python allow storing and accessing multiple values, and they support operations like indexing, slicing, and matrices.
13. Built-in Functions - Python includes built-in functions like `cmp()`, `len()`, `min()`, `max()`, and list comprehension for various tasks.

14. Tuples - Tuples are immutable data structures in Python, allowing the storage of ordered collections of values.

15. Built-in Tuple Functions - Python provides functions like `cmp()` and `len()` for working with tuples.

16. Dictionaries - Dictionaries in Python store key-value pairs, and you can access, update, and delete elements within them.

17. Properties of Dictionaries, Built-in Dictionary Functions & Methods, Dictionary Comprehension - Dictionaries have unique keys, built-in methods, and support dictionary comprehension.

18. Date & Time - Python offers time-related modules, like the `time` module, `calendar` module, and `time` module, for handling dates and times.

19. Functions - Python functions can be defined, called, and can accept arguments, including anonymous functions and return statements.

20. Scope of Variables - Local & Global, Decorators, and Recursion - Python handles variable scope, supports decorators, and allows recursion.

21. Map, Reduce, and Filter - Python functions like `map()`, `reduce()`, and `filter()` help manipulate collections of data.

22. Import Statements, Locating Modules - Python imports modules and libraries using import statements and searches for them in the current directory or `Pythonpath`.

23. `Dir()` Function, Global and Location Functions, `Reload()` Function, `Sys` Module, and `Subprocess` Module - Python includes functions like `dir()` and modules like `sys` and `subprocess` for managing program execution and modules.

24. Packages in Python - Python uses packages to organize and manage modules, making code more maintainable.

25. Files in Python - Python provides methods to read keyboard input, open and close files, and specify file modes.

26. Files Object Attributes - You can use file object attributes like `open`, `close`, and manage file position when reading and writing files.

27. Renaming and Deleting Files - Python supports renaming and deleting files using appropriate functions.

28. Pickle and JSON - Python allows serialization and deserialization of data using libraries like pickle and JSON.

29. Directory Handling - Python offers methods for creating, changing, getting the current working directory, and removing directories.

30. Exception Handling - Python provides try and except blocks for handling exceptions and errors.

31. Try-Finally Clause and User-Defined Exceptions - You can use try-finally and define your own custom exceptions.

32. Object-Oriented Programming (OOP) Concepts - Python supports OOP principles, including classes, objects, inheritance, method overriding, operator overloading, and data encapsulation.

33. Regular Expressions - Python offers functions like match and search for pattern matching and searching within text.

34. Regular Expression Modifiers and Patterns - Python supports modifiers and patterns for more complex regular expressions.

35. Database Connectivity - Python can interact with databases by creating, inserting, updating, and deleting data, along with error handling for database operations.